

现代编程思想

多元组，结构体与枚举类型

Hongbo Zhang

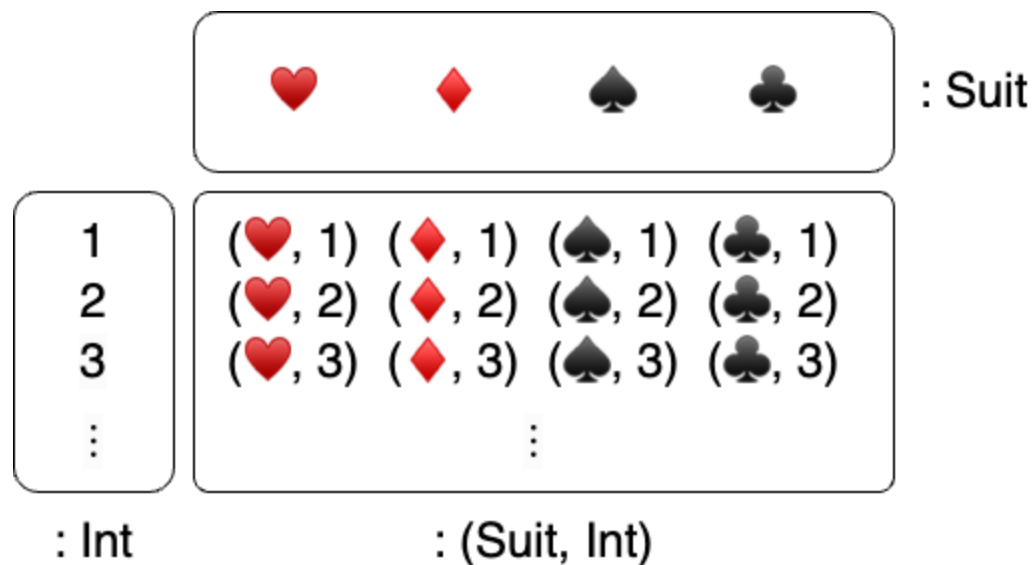
基础数据类型： 多元组与结构体

回顾：多元组

- 多元组：固定长度的不同类型数据的集合
 - 定义：(`<表达式>`, `<表达式>`, ...)
 - 类型：(`<表达式类型>`, `<表达式类型>`, ...)
 - 例如：
 - 身份信息：(`"Bob"`, `2023`, `10`, `24`): (`String`, `Int`, `Int`, `Int`)
 - 成员访问：
 - `<多元组>.<索引>` : `(2023, 10, 24).0 == 2023`
- 列表：任意长度的相同类型数据的集合
 - 例如：
 - 字符的序列：`Cons('H', Cons('i', Cons('!', Nil)))`
 - `Cons` : `construct` 的缩写

笛卡尔积

- 一个多元组类型的元素即是每个组成类型的元素构成的有序元素组
 - 集合的笛卡尔积，又称积类型
 - 例：扑克牌的所有花色： $\{\heartsuit \spadesuit \clubsuit \diamondsuit\} \times \{n \in \mathbb{N} | 1 \leq n \leq 52\}$



结构体

- 元组的问题在于，难以理解其所代表的
 - `(String, Int)`：一个人的姓名和年龄？ 姓名和手机号？ 地址和邮编？
 - 结构体允许我们赋予名称
 - `struct PersonalInfo { name : String; age : Int }`
 - `struct ContactInfo { name : String; telephone : Int }`
 - `struct AddressInfo { address : String; postal : Int }`
- 通过名称，我们能明确数据的信息以及对应字段的含义

结构体的定义

- 结构体的定义形如 `struct <结构体名称> { <字段名> : <类型> ; ... }`
 - `struct PersonalInfo { name : String; age : Int }`
- 定义结构体的值时，形如 `{ <字段名> : <值> , ... }`
 - `let info : PersonalInfo = { name : "Moonbit", age : 1, }`
 - 结构体的值的定义不在意顺序： `{ age : 1, name : "Moonbit", }`
- 如遇到字段名相同的定义无法区分时，可在后面加上类型声明以作区分
 - `struct A { val : Int }`
 - `struct B { val : Int }`
 - `let x = ( { val : 1, } : A )`

结构体的访问与更新

- 访问结构体时，我们通过 `<结构体>.<字段名>`

```
1. let old_info : PersonalInfo = { name : "Moonbit", age : 1, }  
2. let a : Int = old_info.age // 1
```

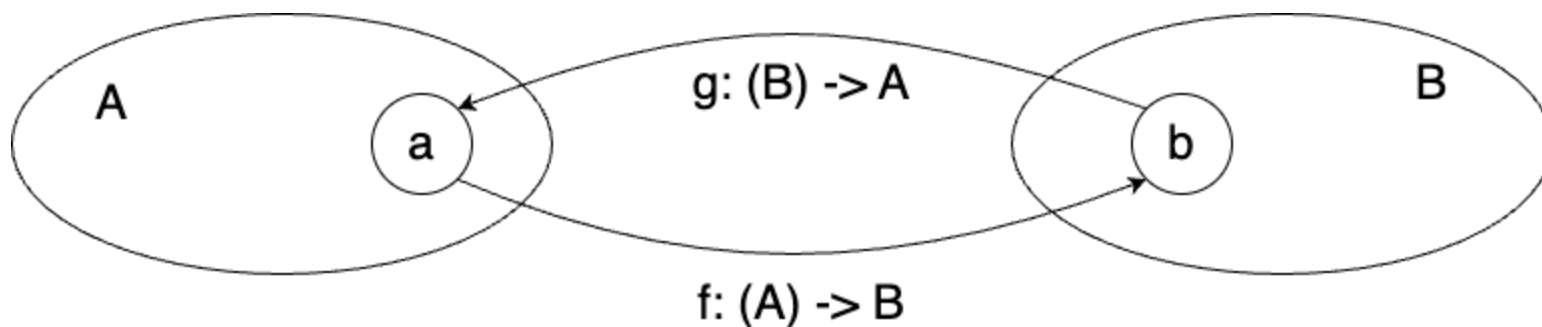
- 更新原有的结构体时，我们可以复用原有的部分，如

```
1. let new_info = { .. old_info, age : 2, }  
2. let other_info = { .. old_info, name : "Hello", }
```

多元组与结构体的关系

- 结构体与相同类型集合构成的多元组同构
 - 集合 `A` 与 `B` 之间存在一一映射的关系
 - 存在一对映射 `f : (A) -> B` 与 `g : (B) -> A` 使得
 - `g(f(a)) == a`
 - `f(g(b)) == b`
- 例: `struct PersonalInfo { name: String; age: Int }` 与 `(String, Int)` 同构

```
1. fn f(info : PersonalInfo) -> (String, Int) { (info.name, info.age) }  
2. fn g(pair : (String, Int)) -> PersonalInfo { { name: pair.0, age: pair.1, } }
```



多元组与结构体的关系

- 多元组是 structural: 只要结构相同 (字段类型一一对应) 就类型兼容

```
1. fn accept_tuple(tuple : (Int, String)) -> Bool {  
2.   true  
3. }  
4. let accepted : Bool = accept_tuple((1, "Yes"))
```

- 结构体是 nominal: 只有类型名相同 (字段顺序可以打乱) 才类型兼容

```
1. struct A { val : Int ; other : Int }  
2. struct B { val : Int ; other : Int }  
3. fn accept_a(a : A) -> Bool {  
4.   true  
5. }  
6. let not_accepted : Bool = accept_a(({ val : 1, other : 2 } : B)) // DO NOT COMPILE  
7. let accepted : Bool = accept_a(({other : 2, val : 1} : A))
```

模式匹配

- 回顾：我们可以通过模式匹配查看列表和Option的结构

```
1. fn head_opt(list : List[Int]) -> Option[Int] {  
2.     match list {  
3.         Nil => None  
4.         Cons(head, _tail) => Some(head)  
5.     }  
6. }
```

```
1. fn get_or_else(option_int : Option[Int], default : Int) -> Int {  
2.     match option_int {  
3.         None => default  
4.         Some(value) => value  
5.     }  
6. }
```

- 模式匹配可以匹配值（逻辑值、数字、字符、字符串）或者构造器

```
1. fn is_zero(i : Int) -> Bool {  
2.     match i {  
3.         0 => true  
4.         1 | 2 | 3 => false  
5.         _ => false  
6.     }  
7. }
```

- 构造器中可以嵌套模式进行匹配，或定义标识符绑定对应结构

```
1. fn contains_zero(l : List[Int]) -> Bool {  
2.     match l {  
3.         Nil => false  
4.         Cons(0, _) => true  
5.         Cons(_, tl) => contains_zero(tl)  
6.     }  
7. }
```

多元组和结构体的模式匹配

- 多元组模式匹配需数量一一对应

```
1. fn first(pair : (Int, Int)) -> Int {  
2.     match pair {  
3.         (first, second) => first  
4.     }  
5. }
```

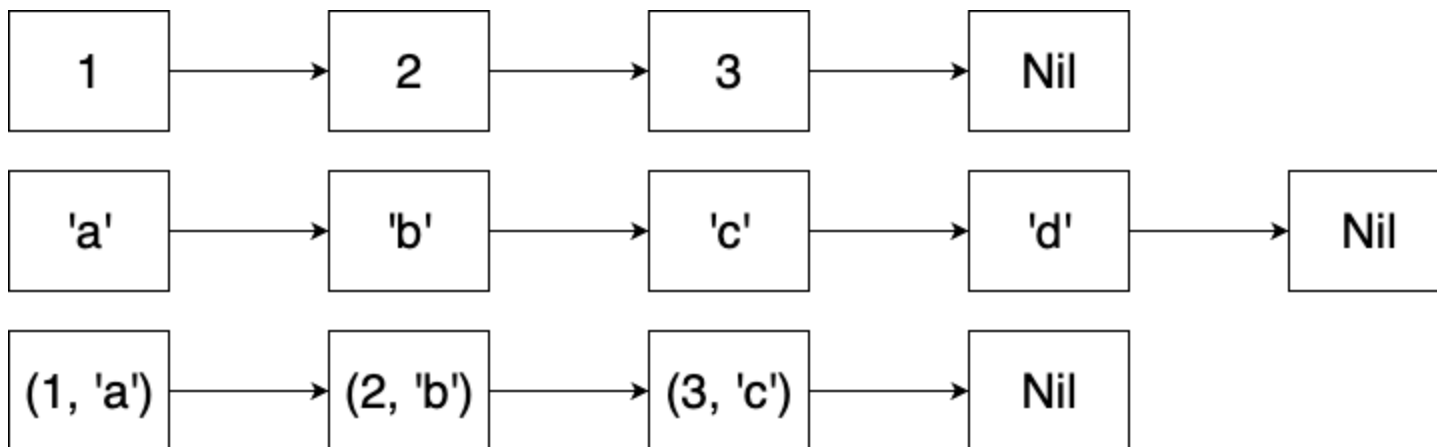
- 结构体模式匹配可以匹配部分字段；可以不用另外命名标识符

```
1. fn baby_name(info : PersonalInfo) -> String? {  
2.     match info {  
3.         { age: 0, .. } => None  
4.         { name, age } => Some(name)  
5.     }  
6. }
```

缝合列表

我们试图缝合两个列表，生成一个数字与字符的二元组的列表，以最短者为准

```
1. fn zip(l1 : List[Int], l2 : List[Char]) -> List[(Int, Char)] {  
2.   match (l1, l2) {  
3.     (Cons(hd, tl), Cons(hd2, tl2)) => Cons((hd, hd2), zip(tl, tl2))  
4.     _ => Nil  
5.   }  
6. }
```



缝合列表

需要注意到模式匹配的顺序是从上到下的

```
1. fn zip(l1 : List[Int], l2 : List[Char]) -> List[(Int, Char)] {  
2.   match (l1, l2) {  
3.     _ => Nil  
4.     // 编辑器会提示未使用的模式及无法抵达的代码  
5.     (Cons(hd, tl), Cons(hd2, tl2)) => Cons((hd, hd2), zip(tl, tl2))  
6.   }  
7. }
```

本地定义中的匹配

我们还可以在本地定义中利用模式进行匹配

- `let <模式> = <表达式>`

此时会根据模式将表达式的值的子结构绑定到定义的标识符上，如：

- `let (first, second) = (1, 2) // first == 1, second == 2`
- `let Cons(1, x) = List::Cons(1, Nil) // x == Nil`
- `let Cons(2, x) = List::Cons(1, Nil) // 运行时错误，程序中止`

枚举类型

不同情况的并集

- 如何定义周一到周日的集合？
- 如何定义硬币落下结果的集合？
- 如何定义表示整数四则运算的结果的集合？
- ...

枚举类型

为了表示不同情况的数据结构，我们使用枚举类型

```
1. enum DaysOfWeek {  
2.     Monday; Tuesday; Wednesday; Thursday; Friday; Saturday; Sunday  
3. }
```

```
1. enum Coin {  
2.     Head  
3.     Tail  
4. }
```

枚举类型的定义与构造

```
1. enum DaysOfWeek {  
2.     Monday; Tuesday; Wednesday; Thursday; Friday; Saturday; Sunday  
3. }
```

- 每一种可能的情况即是构造器

```
1. let monday : DaysOfWeek = Monday  
2. let tuesday : DaysOfWeek = Tuesday
```

- 枚举类型定义可能重复，需要加上 `<类型>::` 加以区分

```
1. enum Repeat1 { A; B }  
2. enum Repeat2 { A; B }  
3. let x : Repeat1 = Repeat1::A
```

枚举类型的意义

- 对比一下两个函数，枚举类型可以与现有类型区分开，更好地实现抽象

```
1. fn tomorrow(today : Int) -> Int
2. fn tomorrow(today : DaysOfWeek) -> DaysOfWeek
3. let tuesday = 1 * 2 // 这是周二吗?
```

- 禁止不合理数据的表示

```
1. struct UserId { email : Option[String]; telephone : Option[Int] }
2. enum UserId {
3.     Email(String)
4.     Telephone(Int)
5. }
```

枚举类型

- 枚举类型的每一种情况也可以承载数据，如

```
1. enum Option[T] {  
2.     Some(T)  
3.     None  
4. }
```

```
1. enum ComputeResult {  
2.     Success(Int)  
3.     Overflow  
4.     DivideByZero  
5. }
```

- 此时枚举类型对应可区分的并集，又称和类型
 - $Option(T) = Some(T) \sqcup \{None\}$

代数数据类型

代数数据类型

我们将多元组、结构体、枚举类型等称为代数数据类型，它们具有代数结构

- 类型相等：同构
- 类型相乘：积类型
- 类型相加：和类型
- 加法的单位元： `enum Nothing {}`
- 乘法的单位元： `() : Unit`

- $1 \times n = n$
 - 对于任意类型 `T`, `(T, Unit)` 与 `T` 同构

```
1. fn[T] f1(t : T) -> (T, Unit) { (t, ()) }
2. fn[T] g1(pair : (T, Unit)) -> T { pair.0 }
```

- $0 + n = n$
 - 对于任意类型 `T`, `enum PlusZero[T] { CaseT(T); CaseZero(Nothing) }` 与 `T` 同构

```
1. fn[T] f2(t : PlusZero) -> T {
2.     match t {
3.         CaseT(t) => t
4.         CaseZero(_) => abort("对应集合为空, 即不存在这样的值")
5.     }
6. }
7.
8. fn[T] g2(t : T) -> PlusZero { CaseT(t) }
```


代数数据类型

- `enum Coins { Head; Tail }`
 - $\text{Coins} = 1 + 1 = 2$
- `enum DaysOfWeek { Monday; Tuesday; ...; }`
 - $\text{DaysOfWeek} = 1 + 1 + 1 + 1 + 1 + 1 + 1 = 7$
- `List` 的定义 (以 `List[Int]` 为例):

$$\begin{aligned}
 \text{enum List} &= \text{Nil} + \text{Int} \times \text{List} \\
 &= 1 + \text{Int} \times \text{List} \\
 &= 1 + \text{Int} \times (1 + \text{Int} \times \text{List}) \\
 &= 1 + \text{Int} \times 1 + \text{Int} \times \text{Int} \times \text{List} \\
 &= 1 + \text{Int} + \text{Int} \times \text{Int} + \text{Int} \times \text{Int} \times \text{Int} + \dots
 \end{aligned}$$

总结

- 本章节介绍了月兔中的诸多自定义数据类型，包括
 - 多元组
 - 结构体
 - 枚举类型并介绍了代数数据类型的概念
- 推荐阅读
 - Category Theory for Programmers 第六章